

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Agile Development Process and Devops Lab

Assignment -9

NAME = Pallab Das

REGISTRATION NUMBER = 24MIS0212

COURSE CODE = ISWE406P

COURSE NAME = Agile Development Process and Devops Lab

FACULTY NAME = Sweta Bhattacharya

Question 1:

Question 1 – University Learning Management Portal with Versioned Releases and Automated Rollback.
Develop an HTML/CSS-based University Learning Management Portal that displays course information, faculty profiles, learning resources, assignment announcements, and examination schedules.
The university wants to automate application releases using a CI/CD pipeline. Every change committed to GitHub must trigger a Jenkins Pipeline that builds and tests a Docker image, publishes the image to Docker Hub, and deploys the application to Kubernetes. The university also requires versioned releases, deployment health checks, and automatic rollback whenever a new release fails.
You are appointed as the DevOps engineer responsible for automating this entire process.

Perform the following:

1. Create a Git repository containing:
    - index.html
    - Dockerfile
    - deployment.yaml
2. Create a Docker image using the Dockerfile.
3. Run the Docker container and verify the webpage locally.
4. Push the Docker image to Docker Hub.
5. Enable Kubernetes in Docker Desktop.
6. Create a Kubernetes Deployment with 2 replicas.
7. Configure Jenkins credentials for:
    - Docker Hub
    - Kubernetes configuration
8. Create a Jenkins Pipeline containing all required stages
9. Create a NodePort Service to expose the application.
10. Verify the application using the assigned NodePort.
11. Verify that both Pods are in the Running state using kubectl get pods.

You must also introduce a deliberately faulty application version, identify the failed rollout, automatically roll back to the previous stable release, and demonstrate that the restored application is functional.

Github Url: https://github.com/PDGamerSG/24MIS0212-Lab-Assessment-9

Step 1: Writing the code

University Learning Management Portal was created using HTML and CSS. The source, Dockerfile, Kubernetes files and Jenkinsfile are stored in the lms-portal folder.

Code: deployment.yaml file

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lms-portal
  namespace: assessment9
spec:
  replicas: 2
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 60
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxUnavailable: 0, maxSurge: 1}
  selector:
    matchLabels:
      app: lms-portal
  template:
    metadata:
      labels:
        app: lms-portal
    spec:
      containers:
        - name: lms-portal
          image: pdgamersg/assessment9-lms:v2
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
          resources:
            requests: {cpu: 10m, memory: 16Mi}
            limits: {cpu: 250m, memory: 64Mi}
          readinessProbe:
            httpGet: {path: /healthz, port: 80}
            initialDelaySeconds: 2
            periodSeconds: 3
          livenessProbe:
            httpGet: {path: /healthz, port: 80}
            initialDelaySeconds: 5
            periodSeconds: 10
```

service.yaml

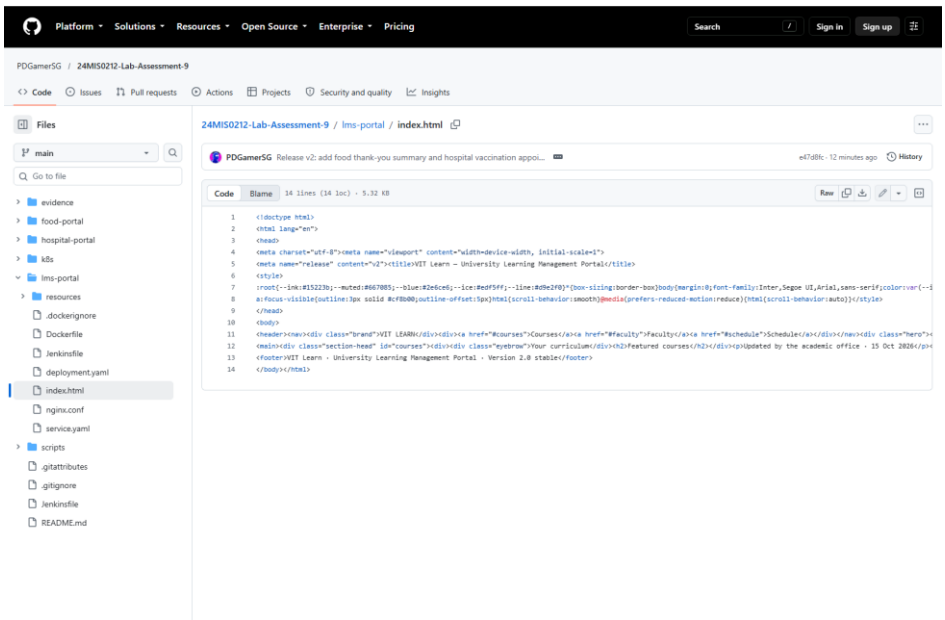
```
apiVersion: v1
kind: Service
metadata:
  name: lms-portal
  namespace: assessment9
spec:
  type: NodePort
  selector:
    app: lms-portal
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30081
```

Dockerfile

```
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY . /usr/share/nginx/html/
EXPOSE 80
HEALTHCHECK --interval=10s --timeout=3s --retries=3 CMD wget -q -O -
http://127.0.0.1/healthz || exit 1
```

index.html

The page displays course information, a faculty profile, a downloadable learning guide, the assessment announcement and the examination schedule.

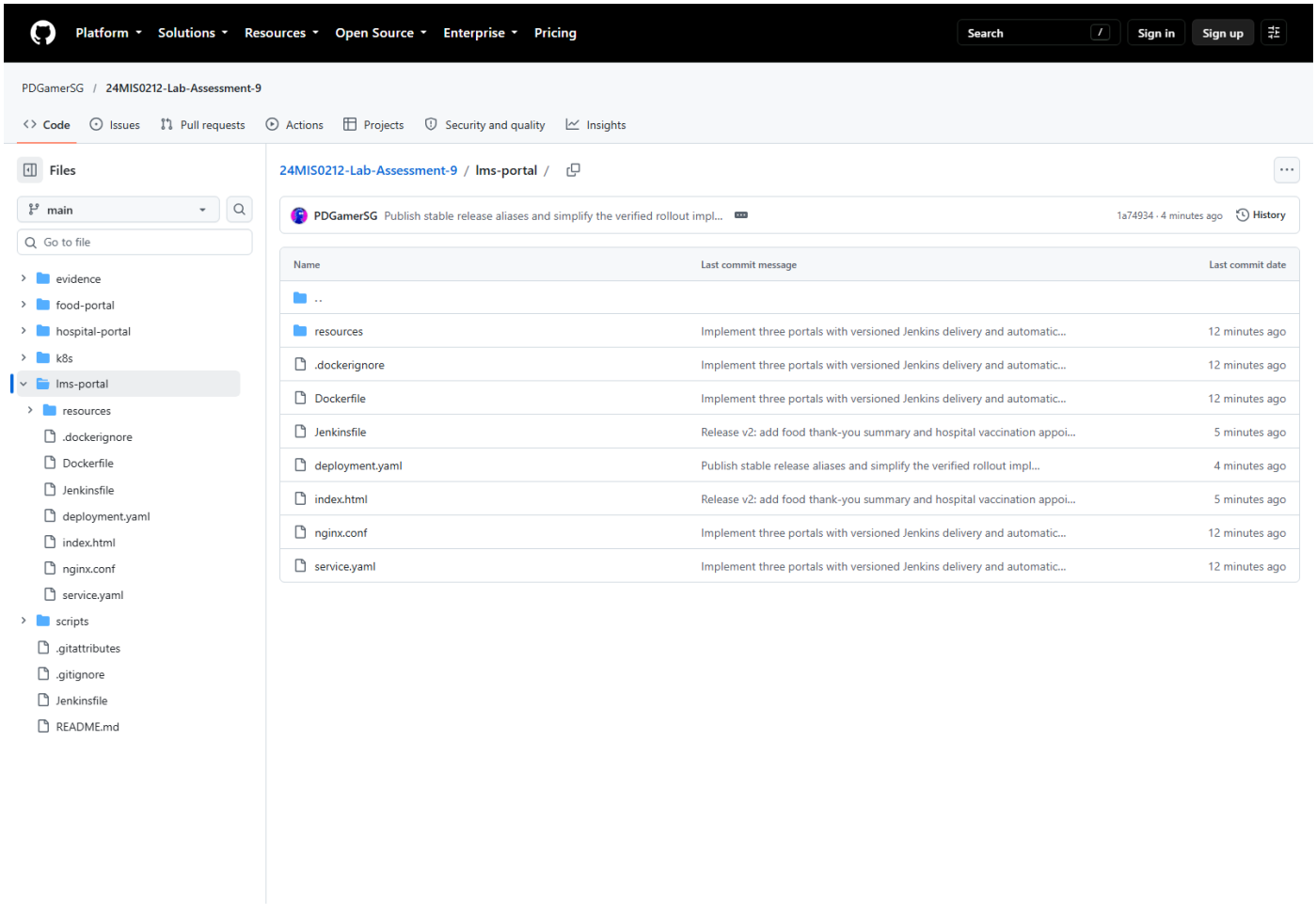


Step 2: Pushing the code to github

One GitHub repository was used for all three questions. The main branch contains the application files and the complete CI/CD implementation.

Commands used:

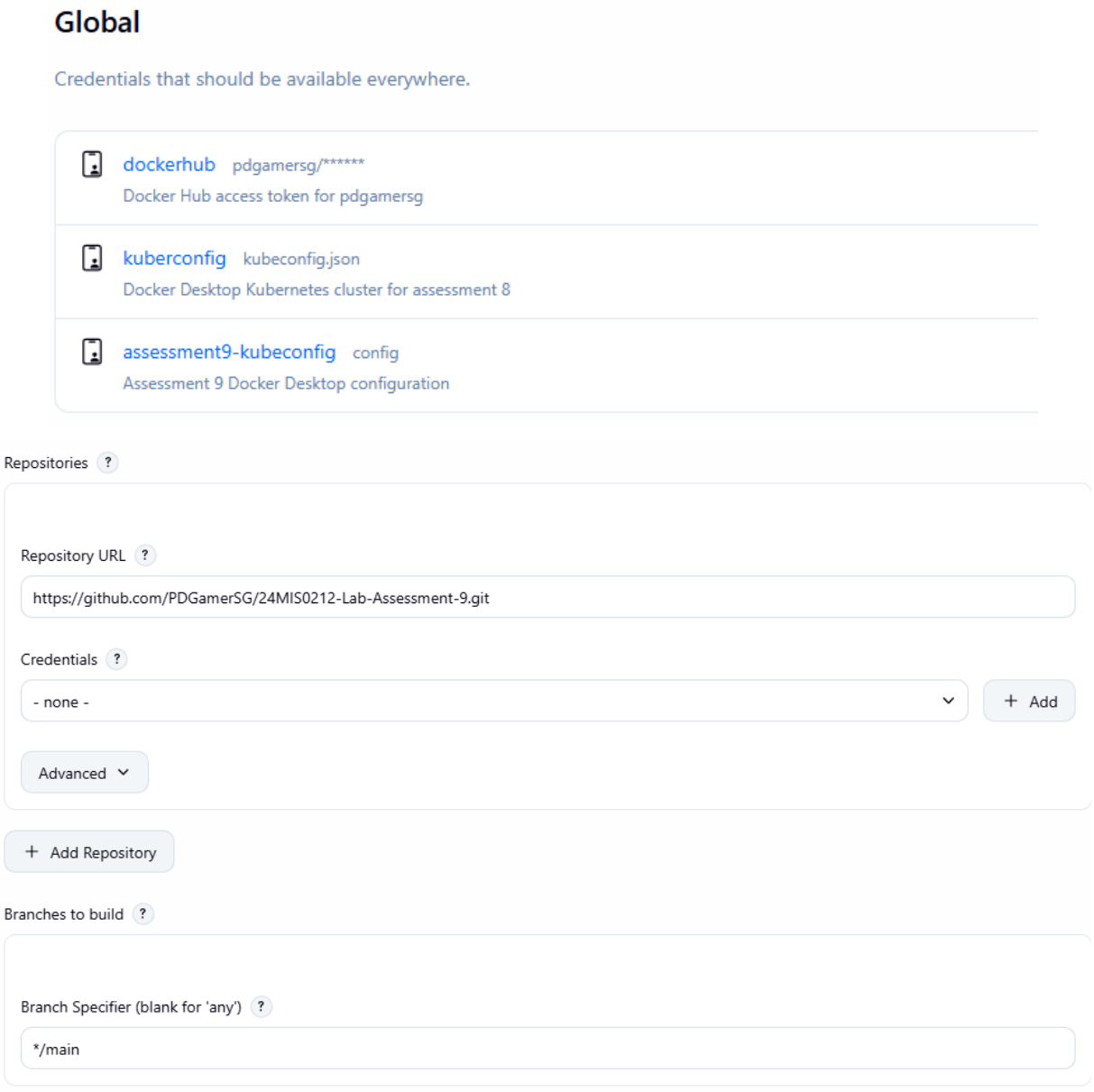
```
git add .
git commit -m "Implement reliable portal delivery"
git push origin main
```



Step 3: Setting up Jenkins

Jenkins uses dockerhub for Docker Hub and assessment9-kubeconfig for Kubernetes. It reads main and polls GitHub every two minutes.

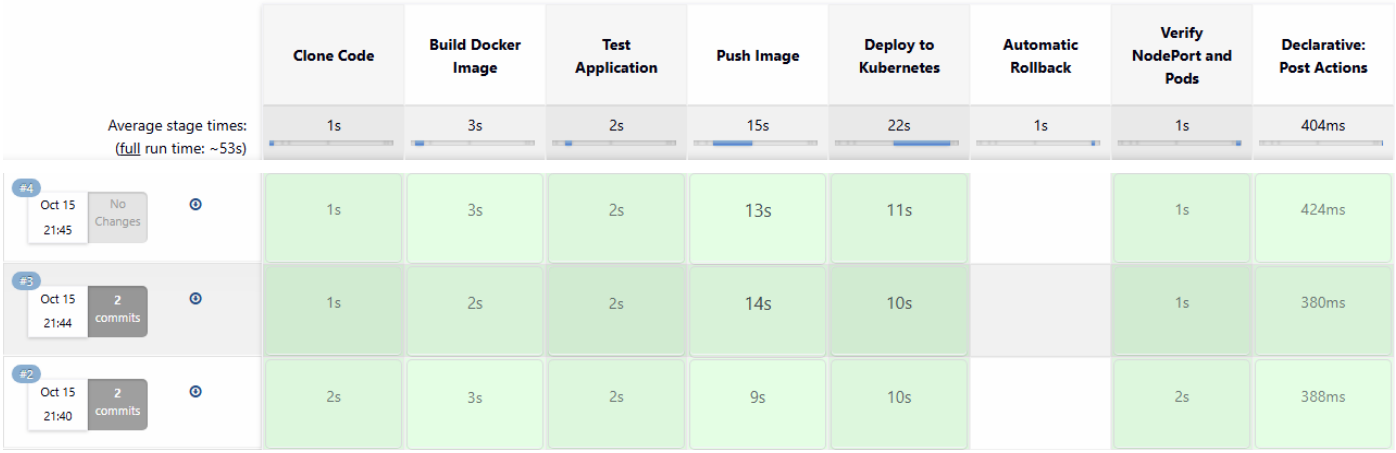
The script path is lms-portal/Jenkinsfile. APP selects the portal, and RELEASE sets the image tag prefix.



Step 4: Stage View

The pipeline clones the code, builds and tests the Docker image, pushes the versioned image to Docker Hub, deploys to Kubernetes and checks the NodePort and Pods.

Assessment9-Q1-LMS - Stage View



	Clone Code	Build Docker Image	Test Application	Push Image	Deploy to Kubernetes	Automatic Rollback	Verify NodePort and Pods	Declarative: Post Actions
Average stage times: (full run time: ~53s)	1s	3s	2s	15s	22s	1s	1s	404ms
#4 Oct 15 21:45 No Changes	1s	3s	2s	13s	11s		1s	424ms
#3 Oct 15 21:44 2 commits	1s	2s	2s	14s	10s		1s	380ms
#2 Oct 15 21:40 2 commits	2s	3s	2s	9s	10s		2s	388ms

The green rows show successful builds #4, #3 and #2. Build #4 uses pdgamersg/assessment9-lms:v2-4.

Local Docker test:

```
$ docker run -d --name assessment9-smoke-lms-4 --restart no -p 127.0.0.1:8191:80 pdgamersg/assessment9-lms:v2-4
508791d1d5c0ef88204292ac1e7f5b88a1476af8349c8d4bb28592f6d5c2177b

$ docker exec assessment9-smoke-lms-4 nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

PASS: HTTP 200, valid nginx config, source SHA-256 40bd3d00ef2affe6f7505390e8a562da77c5ce0e65678487515675b536869543
$ docker rm -f assessment9-smoke-lms-4
assessment9-smoke-lms-4
```

Step 5: Output

Docker Desktop Kubernetes was enabled. The application uses 2 serving replicas, readiness and liveness checks, and NodePort 30081.

```
$ kubectl --context docker-desktop -n assessment9 get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
food-portal-blue    3/3     3            3           5m48s
food-portal-green   3/3     3            3           103s
hospital-portal     3/3     3            3           27m
lms-portal          2/2     2            2           27m

$ kubectl --context docker-desktop -n assessment9 get pods -l app=lms-portal -o wide
NAME                          READY   STATUS      RESTARTS   AGE    IP            NODE                    NOMINATED NODE
READINESS GATES
lms-portal-5b87848545-pvq5h   0/1     Completed   0          104s   10.244.0.32   desktop-control-plane   <none>
<none>
lms-portal-864f89777c-7x7vh   1/1     Running     0          11s    10.244.0.41   desktop-control-plane   <none>
<none>
lms-portal-864f89777c-tpgbx   1/1     Running     0          7s     10.244.0.42   desktop-control-plane   <none>
<none>

$ kubectl --context docker-desktop -n assessment9 get service lms-portal -o wide
NAME         TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE   SELECTOR
lms-portal   NodePort   10.96.224.59   <none>        80:30081/TCP   27m   app=lms-portal

$ docker ps -a --filter name=^/assessment9-lms-nodeport$ --format {{.Names}}
assessment9-lms-nodeport

$ docker start assessment9-lms-nodeport
assessment9-lms-nodeport

PASS: NodePort 30081 HTTP 200; 2 Running/Ready serving Pods; HTML matches Git source.
```

Docker Hub push and image digest:

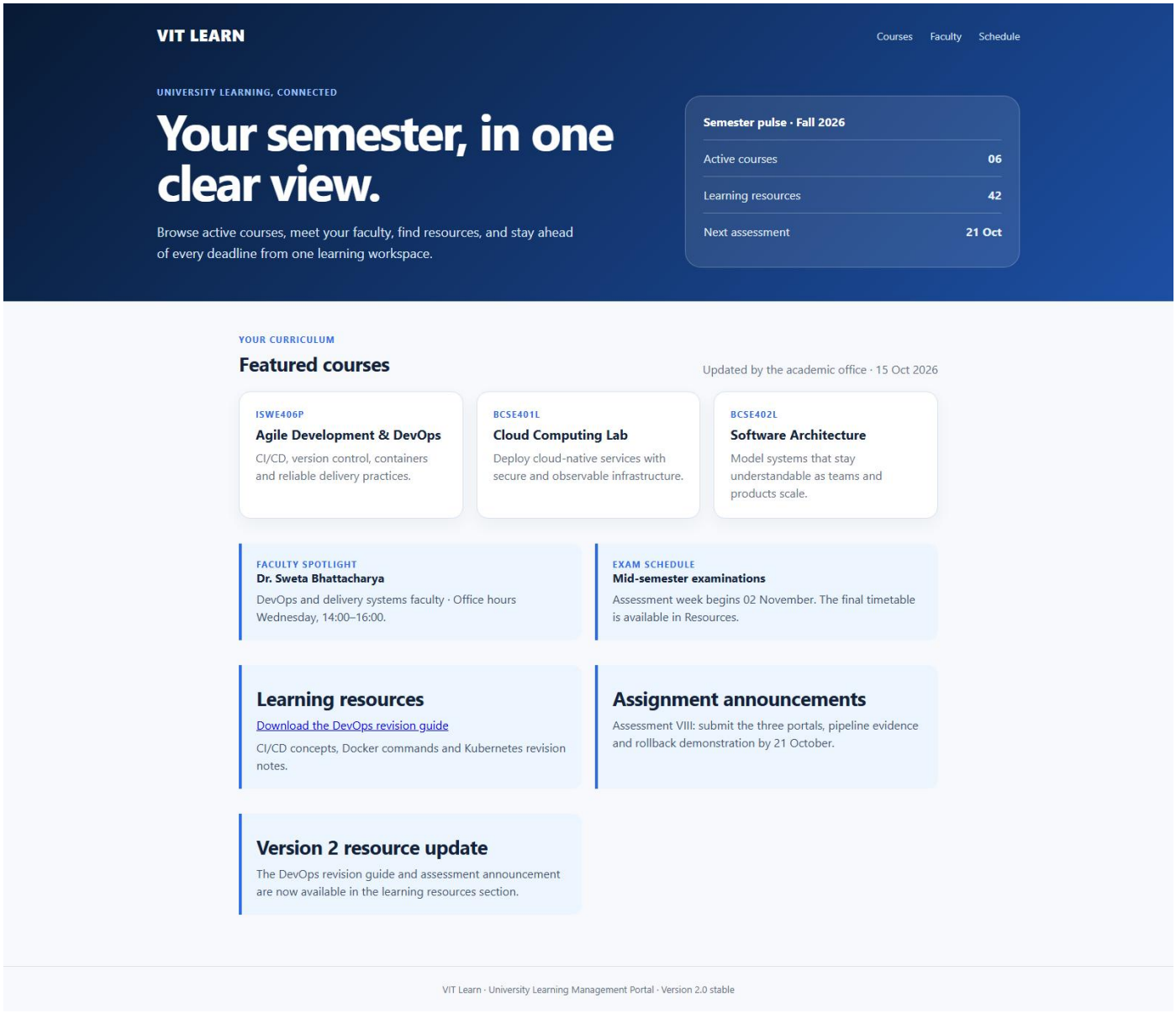
```
2a8b8b0c6568: Waiting
01b0f2948a2d: Layer already exists
197eb75867ef: Layer already exists
b464cfdf2a63: Layer already exists
f18232174bc9: Layer already exists
34a64644b756: Layer already exists
2a8b8b0c6568: Waiting
2a8b8b0c6568: Waiting
2a8b8b0c6568: Waiting
2a8b8b0c6568: Pushed
v2-4: digest: sha256:27dd6e6d45c6d8e221eaac6ac7545eac9db11fa34fbf4163e39553b85e2bf1c9 size: 856

$ docker tag pdgamersg/assessment9-lms:v2-4 pdgamersg/assessment9-lms:v2

$ docker push pdgamersg/assessment9-lms:v2
The push refers to repository [docker.io/pdgamersg/assessment9-lms]
2a8b8b0c6568: Waiting
f18232174bc9: Waiting
81bd8ed7ec67: Waiting
61ca4f733c80: Waiting
029c04b4c8a6: Waiting
d7e507024086: Waiting
34a64644b756: Waiting
```

Step 6: Html output

The application was verified through http://localhost:30081. The NodePort returned HTTP 200, and the served HTML matched the checked-out Git source.

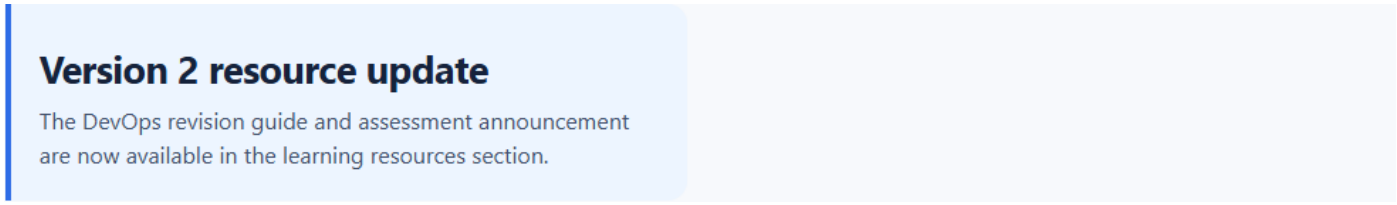


Step 7: Versioned release and health checks

Each Jenkins build publishes a unique version tag. Kubernetes retains five deployment revisions. The /healthz readiness endpoint prevents an unhealthy container from receiving traffic.

The second release adds the learning resource update. Both LMS Pods were Running and Ready after deployment.

Jenkins build #3 was started by an SCM change and deployed the updated source successfully.



Automatic SCM-triggered build:



Result: The updated application was deployed successfully and verified through its assigned NodePort.

Question 2:

Question 2 – Online Food Ordering Portal with Automated Testing and Blue-Green Deployment
Develop an HTML/CSS-based Online Food Ordering Portal that displays restaurant details, food categories, menu items, prices, availability, and promotional offers.
A food delivery company wants to deploy its application using an automated DevOps pipeline. The company requires automated testing before deployment and a deployment strategy that allows the team to validate a new release before directing users to it. If the new version fails, traffic must continue to reach the previous stable version.
You are appointed as the DevOps engineer responsible for implementing the CI/CD pipeline and blue-green deployment strategy using GitHub, Jenkins, Docker, Docker Hub, and Kubernetes.

Perform the following:

1. Create a Git repository containing the application source, Dockerfile, and Kubernetes deployment file.
2. Create a Docker image using the Dockerfile and test the application locally.
3. Push the image to your Docker Hub repository.
4. Create a Kubernetes Deployment with 3 replicas.
5. Configure Jenkins credentials for:
    - Docker Hub
    - Kubernetes configuration
6. Create a Jenkins Pipeline containing all required stages
7. Create a Kubernetes NodePort Service.
8. Deploy the application on Docker Desktop Kubernetes.
9. Verify the status of the Deployment and all three Pods
10. Modify the HTML application by adding a new feature, such as:
    - Course rating summary, or
    - Faculty profile section, or
    - Thank-you message.
11. Build a new Docker image after making the modification.
12. Push the updated image to Docker Hub.
13. Update the Kubernetes Deployment with the new image.
14. Verify that the updated application is accessible through the NodePort.

You must also introduce a deliberately faulty application version, identify the failed rollout, automatically roll back to the previous stable release, and demonstrate that the restored application is functional.

Github Url: https://github.com/PDGamerSG/24MIS0212-Lab-Assessment-9

Step 1: Writing the code

Online Food Ordering Portal was created using HTML and CSS. The source, Dockerfile, Kubernetes files and Jenkinsfile are stored in the food-portal folder.

Code: deployment.yaml file

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: food-portal-blue
  namespace: assessment9
spec:
  replicas: 3
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 60
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxUnavailable: 0, maxSurge: 1}
  selector:
    matchLabels:
      app: food-portal
      slot: blue
  template:
    metadata:
      labels:
        app: food-portal
        slot: blue
    spec:
      containers:
        - name: food-portal
          image: pdgamersg/assessment9-food:v2
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
          resources:
            requests: {cpu: 10m, memory: 16Mi}
            limits: {cpu: 250m, memory: 64Mi}
          readinessProbe:
            httpGet: {path: /healthz, port: 80}
            initialDelaySeconds: 2
            periodSeconds: 3
          livenessProbe:
            httpGet: {path: /healthz, port: 80}
            initialDelaySeconds: 5
            periodSeconds: 10
```

service.yaml

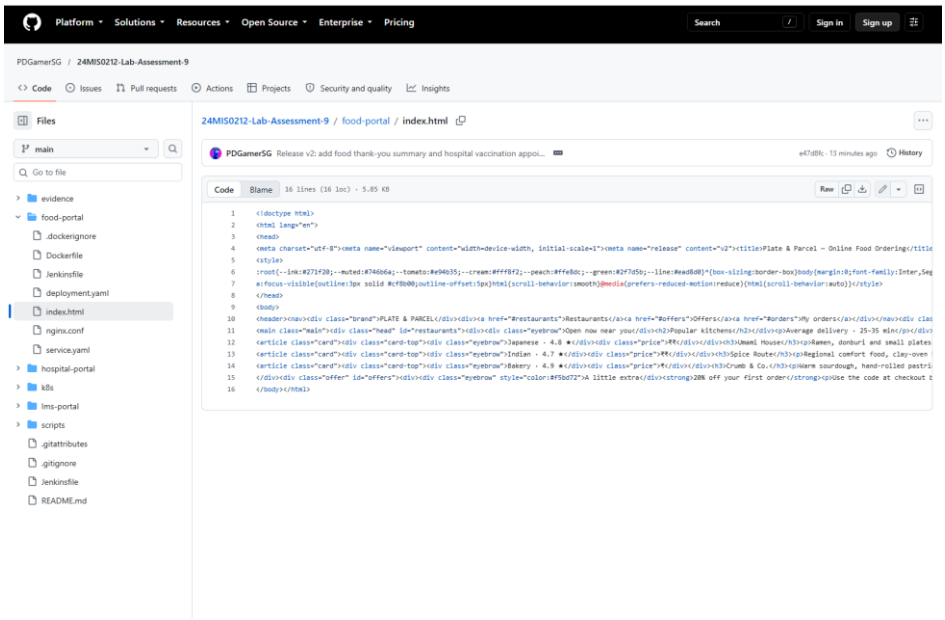
```
apiVersion: v1
kind: Service
metadata:
  name: food-portal
  namespace: assessment9
spec:
  type: NodePort
  selector:
    app: food-portal
    slot: blue
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30082
```

Dockerfile

```
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY . /usr/share/nginx/html/
EXPOSE 80
HEALTHCHECK --interval=10s --timeout=3s --retries=3 CMD wget -q -O -
http://127.0.0.1/healthz || exit 1
```

index.html

The page displays restaurant details, food categories, menu items, exact prices, availability and the WELCOME20 promotional offer.

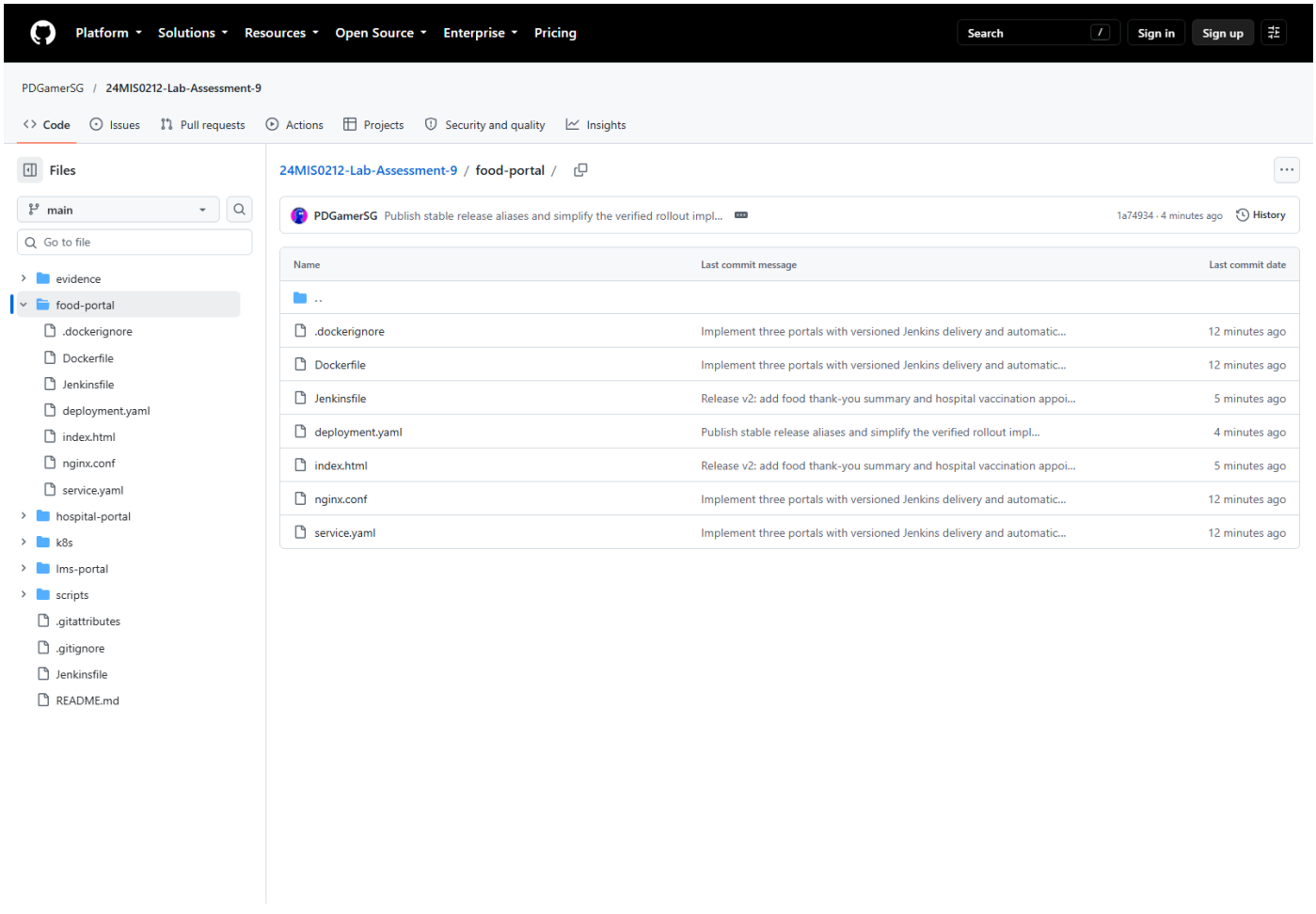


Step 2: Pushing the code to github

One GitHub repository was used for all three questions. The main branch contains the application files and the complete CI/CD implementation.

Commands used:

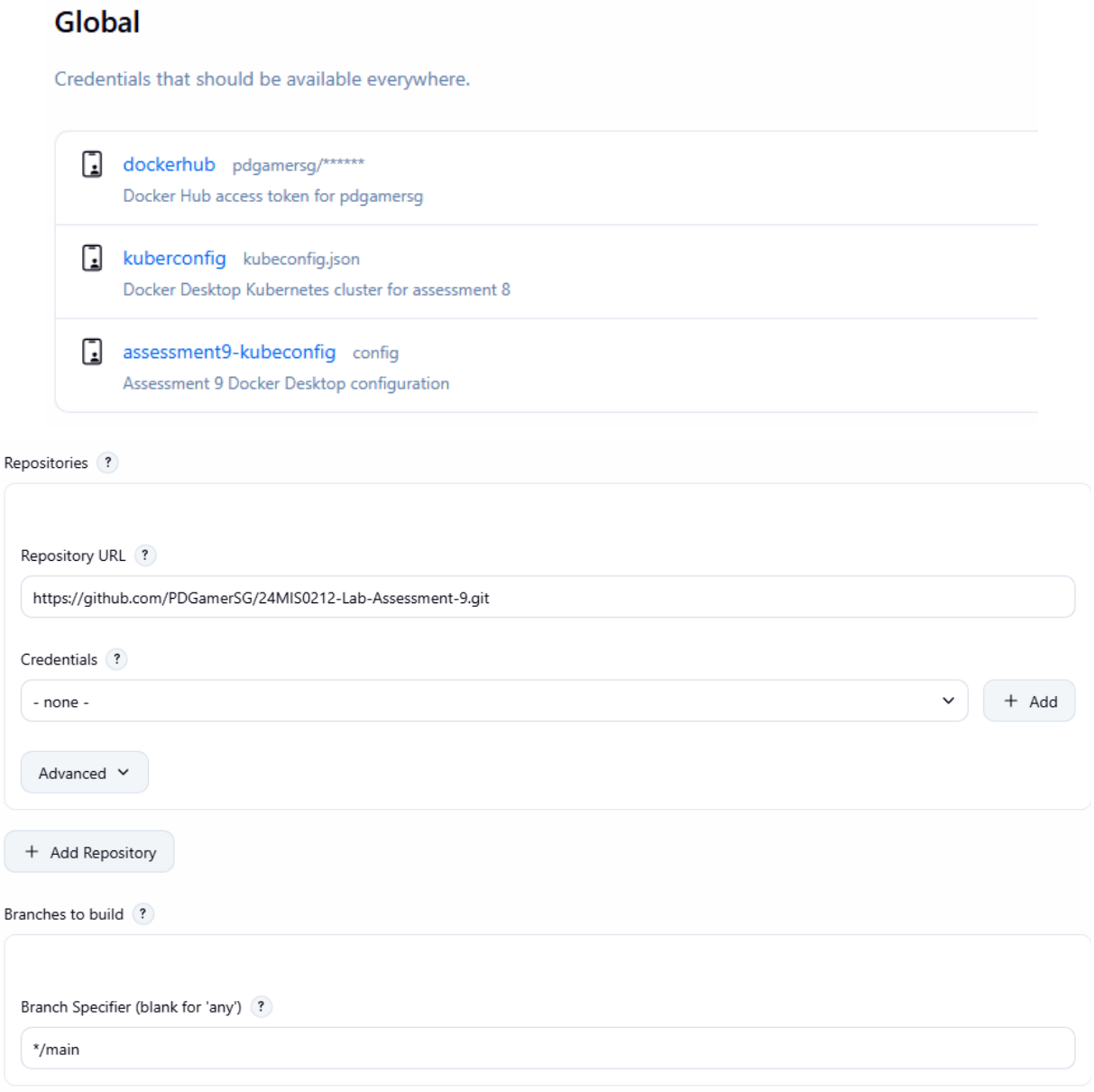
```
git add .
git commit -m "Implement reliable portal delivery"
git push origin main
```



Step 3: Setting up Jenkins

Jenkins uses dockerhub for Docker Hub and assessment9-kubeconfig for Kubernetes. It reads main and polls GitHub every two minutes.

The script path is food-portal/Jenkinsfile. APP selects the portal, and RELEASE sets the image tag prefix.



Step 4: Stage View

The pipeline clones the code, builds and tests the Docker image, pushes the versioned image to Docker Hub, deploys to Kubernetes and checks the NodePort and Pods.

Assessment9-Q2-Food - Stage View

	Clone Code	Build Docker Image	Test Application	Push Image	Deploy to Kubernetes	Automatic Rollback	Verify NodePort and Pods	Declarative: Post Actions
Average stage times: (full run time: ~47s)	1s	3s	2s	11s	20s	764ms	1s	495ms
#4 Oct 15 21:45 No Changes	1s	3s	2s	13s	15s		1s	381ms
#3 Oct 15 21:44 2 commits	1s	3s	2s	15s	7s		1s	398ms
#2 Oct 15 21:40 2 commits	1s	2s	2s	11s	8s		1s	451ms

The green rows show successful builds #4, #3 and #2. Build #4 uses pdgamersg/assessment9-food:v2-4.

Local Docker test:

```
$ docker run -d --name assessment9-smoke-food-4 --restart no -p 127.0.0.1:8192:80 pdgamersg/assessment9-food:v2-4
2f6b526847cfb7bf65b08659496bfebb8b850443b04392d6d40918240d9ae223

$ docker exec assessment9-smoke-food-4 nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

PASS: HTTP 200, valid nginx config, source SHA-256 7a7a0f62905626d7002fc74fb6fb99cd3ffbbe85152518956a6b0becadfa003c
$ docker rm -f assessment9-smoke-food-4
assessment9-smoke-food-4
```

Step 5: Output

Docker Desktop Kubernetes was enabled. The application uses 3 serving replicas, readiness and liveness checks, and NodePort 30082.

```
$ kubectl --context docker-desktop -n assessment9 get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
food-portal-blue    3/3     3            3           6m27s
food-portal-green   3/3     3            3           2m22s
hospital-portal     3/3     3            3           28m
lms-portal          2/2     2            2           28m

$ kubectl --context docker-desktop -n assessment9 get pods -l app=food-portal,slot=blue -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE                    NOMINATED
NODE   READINESS GATES
food-portal-blue-6647dc7f85-6b54t   1/1     Running   0          16s   10.244.0.45   desktop-control-plane   <none>
<none>
food-portal-blue-6647dc7f85-7xckb   1/1     Running   0          11s   10.244.0.46   desktop-control-plane   <none>
<none>
food-portal-blue-6647dc7f85-fl8m9   1/1     Running   0          6s    10.244.0.47   desktop-control-plane   <none>
<none>

$ kubectl --context docker-desktop -n assessment9 get service food-portal -o wide
NAME          TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE   SELECTOR
food-portal   NodePort   10.96.79.197   <none>        80:30082/TCP   28m   app=food-portal,slot=blue

$ docker ps -a --filter name=^/assessment9-food-nodeport$ --format {{.Names}}
assessment9-food-nodeport

$ docker start assessment9-food-nodeport
assessment9-food-nodeport

PASS: NodePort 30082 HTTP 200; 3 Running/Ready serving Pods; HTML matches Git source.
```

Docker Hub push and image digest:

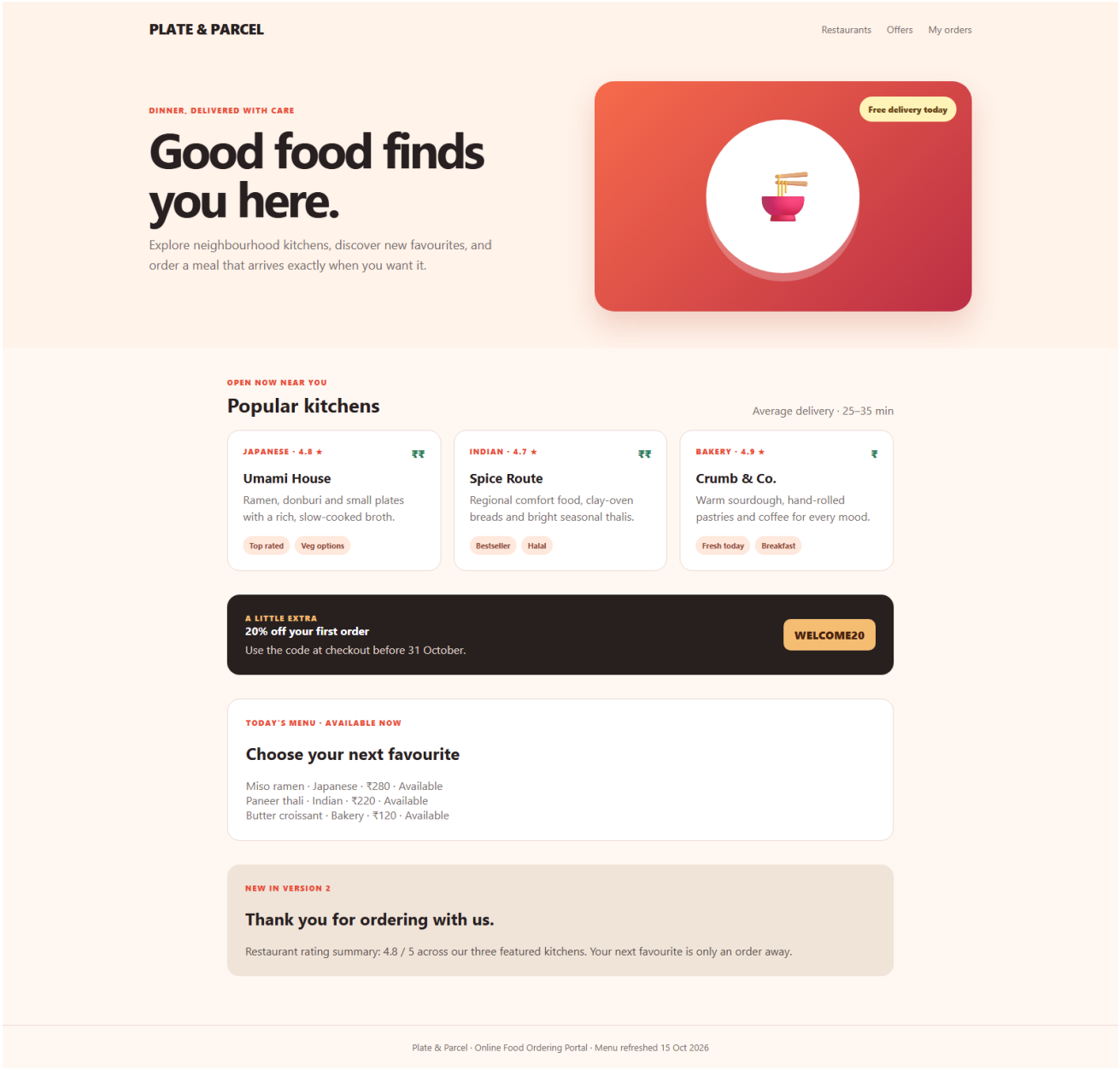
```
81bd8ed7ec67: Waiting
b464cfdf2a63: Waiting
9c61b8ed3fad: Waiting
f18232174bc9: Layer already exists
39c2ddfd6010: Layer already exists
81bd8ed7ec67: Layer already exists
b464cfdf2a63: Layer already exists
9c61b8ed3fad: Waiting
d7e507024086: Layer already exists
9c61b8ed3fad: Pushed
v2-4: digest: sha256:8fe992994a7d24a4d1c0177e0f5e03e684e87fd34ee055bbe6590a8f9cfdfaaf size: 856

$ docker tag pdgamersg/assessment9-food:v2-4 pdgamersg/assessment9-food:v2

$ docker push pdgamersg/assessment9-food:v2
The push refers to repository [docker.io/pdgamersg/assessment9-food]
d7e507024086: Waiting
9c61b8ed3fad: Waiting
f18232174bc9: Waiting
197eb75867ef: Waiting
01b0f2948a2d: Waiting
779d4897f195: Waiting
81bd8ed7ec67: Waiting
```

Step 6: Html output

The application was verified through http://localhost:30082. The NodePort returned HTTP 200, and the served HTML matched the checked-out Git source.



Step 7: Updating the application using blue-green deployment

The updated page adds a thank-you message and a restaurant rating summary. Jenkins deployed the inactive colour with three replicas and waited for its health checks before changing the production Service selector.

Each colour has three Pods. The NodePort selects only the active colour; the other healthy colour remains available as a fallback.

Jenkins build #3 was started by an SCM change and deployed the updated source successfully.



Healthy candidate promoted to blue:

```
food-portal-blue-6647dc7f85-7xckb    1/1     Running       0          10s     10.244.0.46   desktop-control-plane
<none>           <none>
food-portal-blue-6647dc7f85-fl8m9    1/1     Running       0          5s      10.244.0.47   desktop-control-plane
<none>           <none>
food-portal-blue-6fd9dd46db-56d5s    1/1     Terminating   0          6m26s   10.244.0.26   desktop-control-plane
<none>           <none>
food-portal-green-6cd476b456-64swx   1/1     Running       0          2m21s   10.244.0.35   desktop-control-plane
<none>           <none>
food-portal-green-6cd476b456-bvrp2   1/1     Running       0          2m21s   10.244.0.33   desktop-control-plane
<none>           <none>
food-portal-green-6cd476b456-n764t   1/1     Running       0          2m21s   10.244.0.34   desktop-control-plane
<none>           <none>

$ kubectl --context docker-desktop -n assessment9 patch service food-portal --type=merge -p {"spec": {"selector": {"app":
"food-portal", "slot": "blue"}}}
service/food-portal patched

BLUE-GREEN PROMOTION: healthy blue receives production traffic
$ kubectl --context docker-desktop -n assessment9 delete deployment food-portal --ignore-not-found=true

$ docker ps -a --filter name=^/assessment9-food-nodeport$ --format {{.Names}}
assessment9-food-nodeport

$ docker start assessment9-food-nodeport
assessment9-food-nodeport
```

Result: The updated application was deployed successfully and verified through its assigned NodePort.

Question 3:

Question 3 – Online Hospital Appointment Portal with CI/CD Automation and Rollback
Develop an HTML/CSS-based Hospital Appointment Portal that displays doctor profiles, medical specializations, consultation timings, available appointment slots, and hospital announcements.
The hospital wants to automate the deployment of its appointment portal so that every code modification is automatically built, tested, containerized, and deployed. The DevOps team must ensure that defective releases do not remain active and that the previous stable version can be restored automatically when a deployment fails.
You are appointed as the DevOps engineer responsible for implementing the complete CI/CD workflow using GitHub, Docker, Docker Hub, Jenkins, and Kubernetes.

Perform the following:
Implement an automated CI/CD pipeline using GitHub, Jenkins, Docker, Docker Hub, and Kubernetes.
The pipeline should include the following stages:

1. Clone Code – Clone the application from GitHub.
2. Build Docker Image – Build the application image using the Dockerfile.
3. Test Application – Verify that the Docker image can be built successfully.
4. Push Image – Authenticate with Docker Hub and push the image.
5. Deploy to Kubernetes – Use kubectl to deploy the application.
6. Configure Kubernetes with:
    - 3 replicas
    - Kubernetes Deployment
    - NodePort Service
7. Configure Jenkins credentials for:
    - Docker Hub
    - Kubernetes configuration
8. Create a Jenkins Pipeline containing all required stages.
9. Make a modification to the placement webpage in GitHub, such as adding a new company or job opportunity.
10. Trigger the Jenkins pipeline again.
11. Verify that Jenkins automatically:
    - Retrieves the updated code
    - Builds the new Docker image
    - Pushes the image to Docker Hub
    - Deploys the updated application to Kubernetes
12. Verify the updated application through the NodePort.
13. Demonstrate that 3 Kubernetes Pods are running after deployment.

You must also introduce a deliberately faulty application version, identify the failed rollout, automatically roll back to the previous stable release, and demonstrate that the restored application is functional.

Github Url: https://github.com/PDGamerSG/24MIS0212-Lab-Assessment-9

Step 1: Writing the code

Hospital Appointment Portal was created using HTML and CSS. The source, Dockerfile, Kubernetes files and Jenkinsfile are stored in the hospital-portal folder.

Code: deployment.yaml file

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hospital-portal
  namespace: assessment9
spec:
  replicas: 3
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 60
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxUnavailable: 0, maxSurge: 1}
  selector:
    matchLabels:
      app: hospital-portal
  template:
    metadata:
      labels:
        app: hospital-portal
    spec:
      containers:
        - name: hospital-portal
          image: pdgamersg/assessment9-hospital:v2
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
          resources:
            requests: {cpu: 10m, memory: 16Mi}
            limits: {cpu: 250m, memory: 64Mi}
          readinessProbe:
            httpGet: {path: /healthz, port: 80}
            initialDelaySeconds: 2
            periodSeconds: 3
          livenessProbe:
            httpGet: {path: /healthz, port: 80}
            initialDelaySeconds: 5
            periodSeconds: 10
```

service.yaml

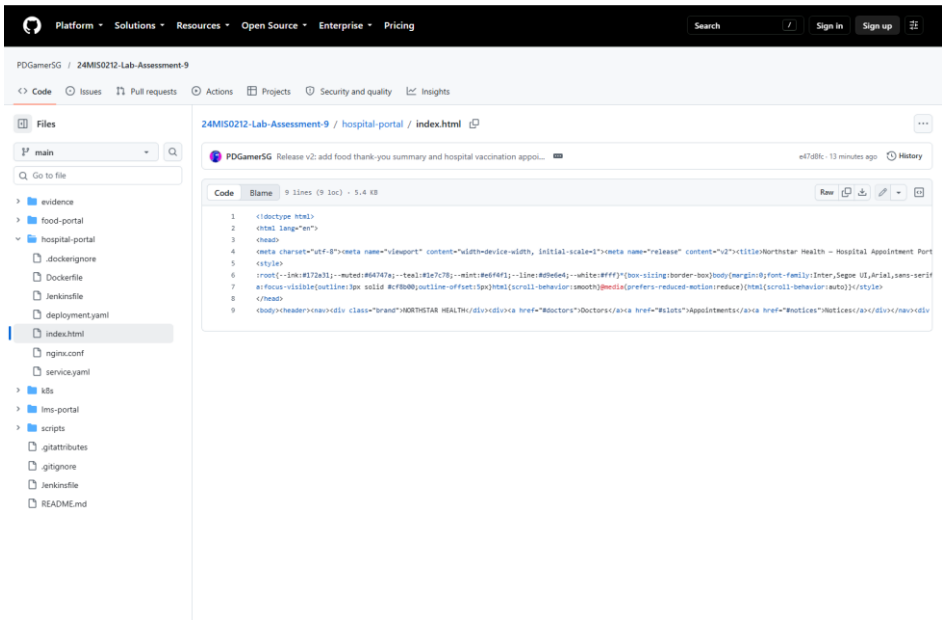
```
apiVersion: v1
kind: Service
metadata:
  name: hospital-portal
  namespace: assessment9
spec:
  type: NodePort
  selector:
    app: hospital-portal
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30083
```

Dockerfile

```
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY . /usr/share/nginx/html/
EXPOSE 80
HEALTHCHECK --interval=10s --timeout=3s --retries=3 CMD wget -q -O -
http://127.0.0.1/healthz || exit 1
```

index.html

The page displays doctor profiles, specialisations, consultation details, appointment slots and hospital announcements.

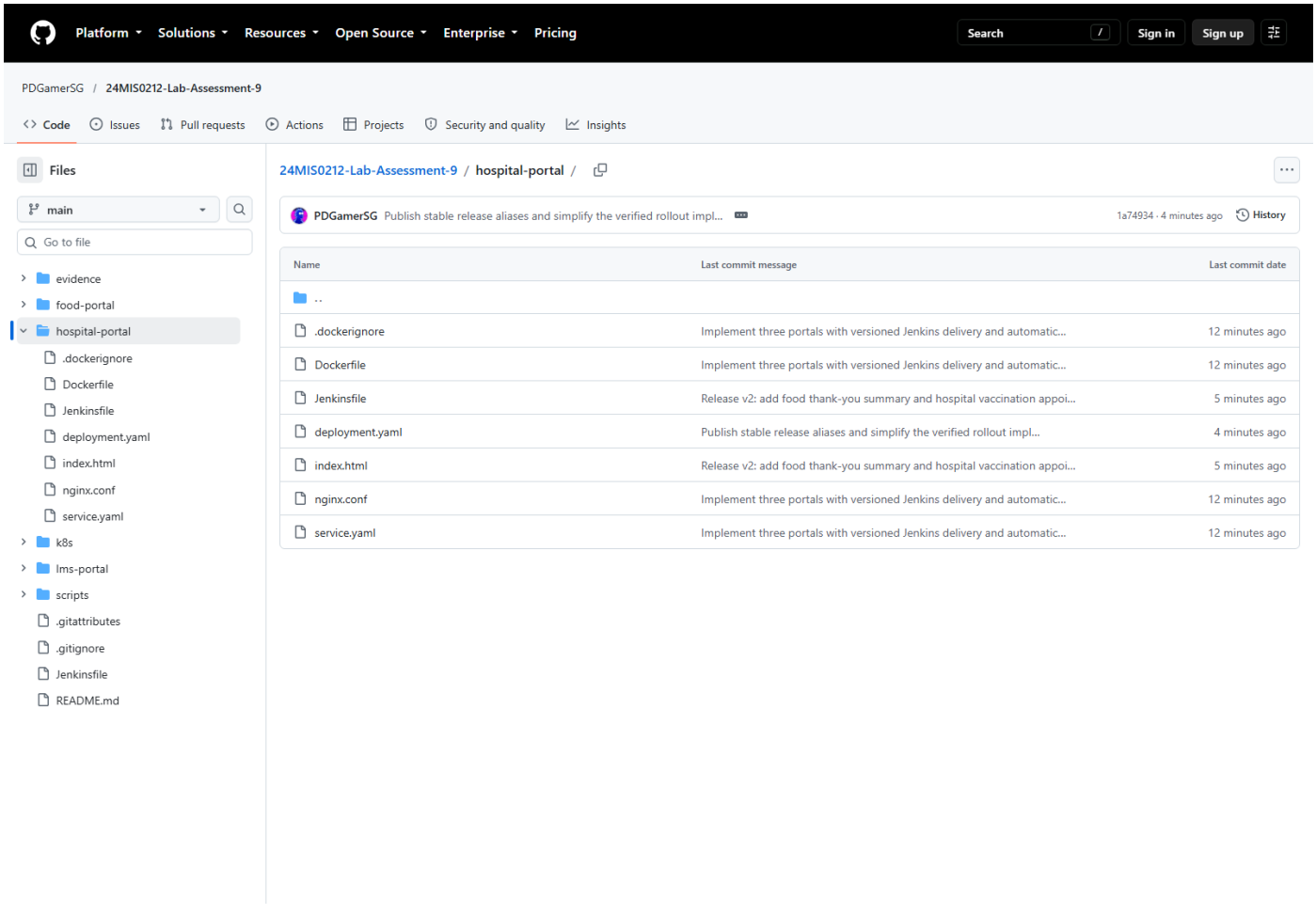


Step 2: Pushing the code to github

One GitHub repository was used for all three questions. The main branch contains the application files and the complete CI/CD implementation.

Commands used:

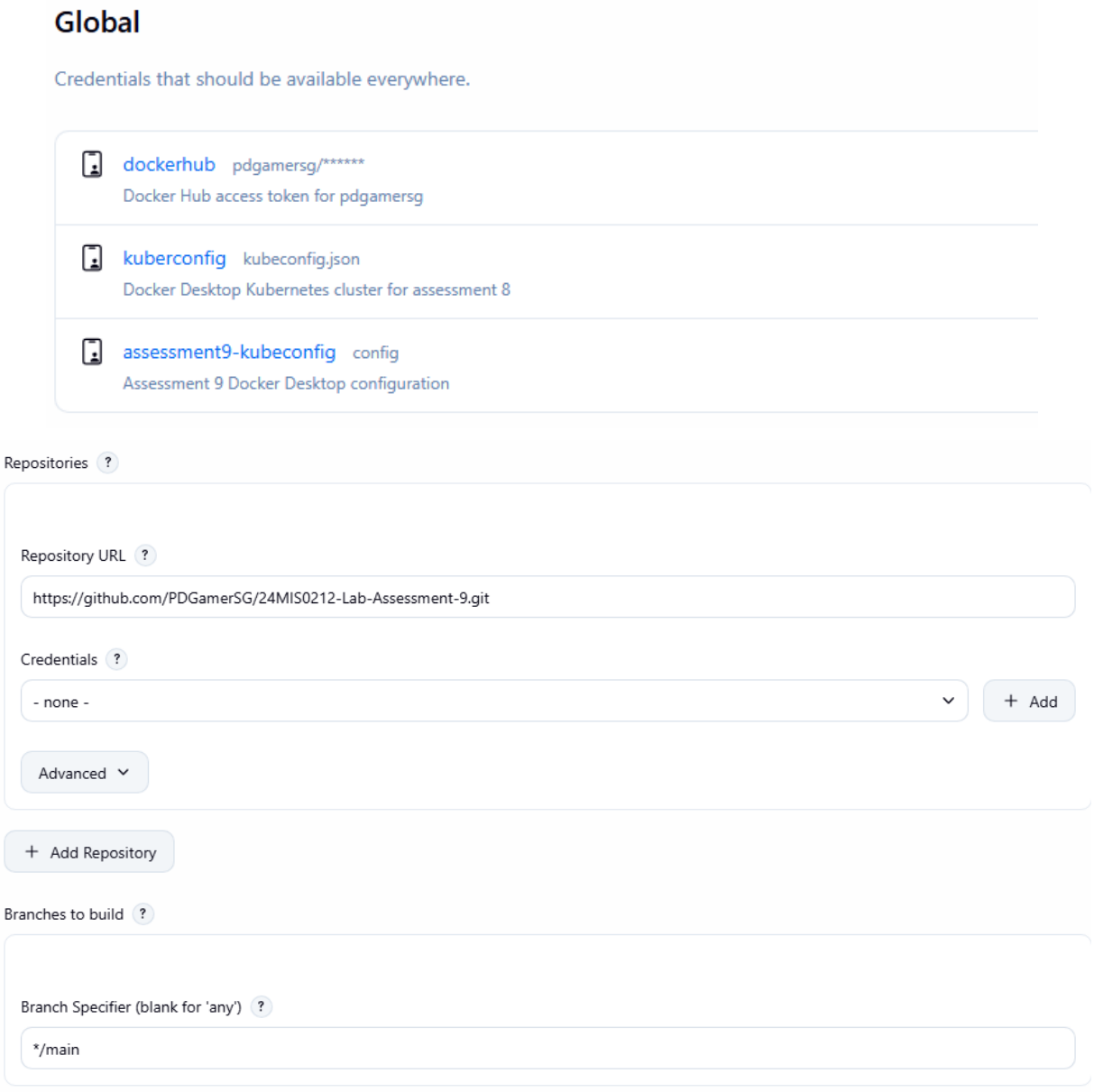
```
git add .
git commit -m "Implement reliable portal delivery"
git push origin main
```



Step 3: Setting up Jenkins

Jenkins uses dockerhub for Docker Hub and assessment9-kubeconfig for Kubernetes. It reads main and polls GitHub every two minutes.

The script path is hospital-portal/Jenkinsfile. APP selects the portal, and RELEASE sets the image tag prefix.



Step 4: Stage View

The pipeline clones the code, builds and tests the Docker image, pushes the versioned image to Docker Hub, deploys to Kubernetes and checks the NodePort and Pods.

Assessment9-Q3-Hospital - Stage View

	Clone Code	Build Docker Image	Test Application	Push Image	Deploy to Kubernetes	Automatic Rollback	Verify NodePort and Pods	Declarative: Post Actions
Average stage times: (full run time: ~53s)	1s	3s	2s	9s	24s	507ms	966ms	489ms
#4 Oct 15 21:45 No Changes	1s	2s	2s	13s	16s		1s	377ms
#3 Oct 15 21:44 2 commits	1s	3s	2s	14s	19s		1s	393ms
#2 Oct 15 21:40 2 commits	1s	3s	2s	9s	17s		1s	493ms

The green rows show successful builds #4, #3 and #2. Build #4 uses pdgamersg/assessment9-hospital:v2-4.

Local Docker test:

```
$ docker run -d --name assessment9-smoke-hospital-4 --restart no -p 127.0.0.1:8193:80 pdgamersg/assessment9-hospital:v2-4
d0be1bf8a7388b4962b9424d52bb395eb3f99f79783235bcf4b8d28ac48ff540

$ docker exec assessment9-smoke-hospital-4 nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

PASS: HTTP 200, valid nginx config, source SHA-256 0c76120344a35b917c4921dab009ed35108753c29010595a2a88191def9933b8
$ docker rm -f assessment9-smoke-hospital-4
assessment9-smoke-hospital-4
```

Step 5: Output

Docker Desktop Kubernetes was enabled. The application uses 3 serving replicas, readiness and liveness checks, and NodePort 30083.

```
$ kubectl --context docker-desktop -n assessment9 get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
food-portal-blue    3/3     3            3           5m53s
food-portal-green   3/3     3            3           108s
hospital-portal     3/3     3            3           27m
lms-portal          2/2     2            2           27m

$ kubectl --context docker-desktop -n assessment9 get pods -l app=hospital-portal -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE                    NOMINATED
NODE   READINESS GATES
hospital-portal-86b9b78669-c2hll    1/1     Running   0          6s    10.244.0.44   desktop-control-plane   <none>
<none>
hospital-portal-86b9b78669-jgpg2    1/1     Running   0          16s   10.244.0.40   desktop-control-plane   <none>
<none>
hospital-portal-86b9b78669-vzd48    1/1     Running   0          12s   10.244.0.43   desktop-control-plane   <none>
<none>

$ kubectl --context docker-desktop -n assessment9 get service hospital-portal -o wide
NAME              TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE   SELECTOR
hospital-portal   NodePort   10.96.20.202   <none>        80:30083/TCP   27m   app=hospital-portal

$ docker ps -a --filter name=^/assessment9-hospital-nodeport$ --format {{.Names}}
assessment9-hospital-nodeport

$ docker start assessment9-hospital-nodeport
assessment9-hospital-nodeport

PASS: NodePort 30083 HTTP 200; 3 Running/Ready serving Pods; HTML matches Git source.
```

Docker Hub push and image digest:

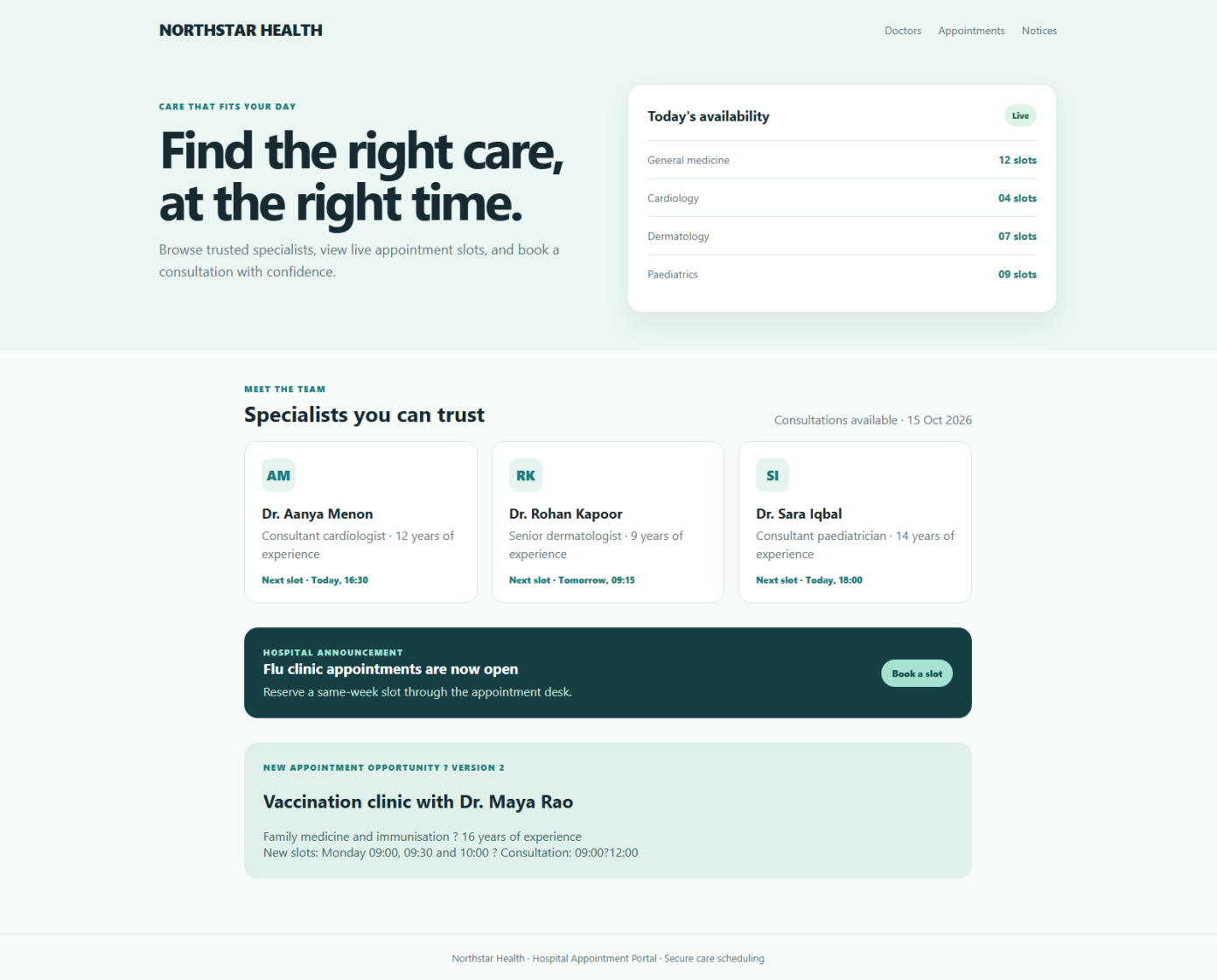
```
bbe60cd3af56: Waiting
81bd8ed7ec67: Waiting
61ca4f733c80: Layer already exists
34a64644b756: Layer already exists
81bd8ed7ec67: Layer already exists
f18232174bc9: Layer already exists
d7e507024086: Layer already exists
197eb75867ef: Layer already exists
bbe60cd3af56: Waiting
bbe60cd3af56: Pushed
v2-4: digest: sha256:e935a2282d899abf1c0d3fa22d6414db977527a8ec4c78ebbdb60b0fd612d5c8 size: 856

$ docker tag pdgamersg/assessment9-hospital:v2-4 pdgamersg/assessment9-hospital:v2

$ docker push pdgamersg/assessment9-hospital:v2
The push refers to repository [docker.io/pdgamersg/assessment9-hospital]
d7e507024086: Waiting
61ca4f733c80: Waiting
39c2ddfd6010: Waiting
01b0f2948a2d: Waiting
197eb75867ef: Waiting
f18232174bc9: Waiting
b464cfdf2a63: Waiting
```

Step 6: Html output

The application was verified through http://localhost:30083. The NodePort returned HTTP 200, and the served HTML matched the checked-out Git source.

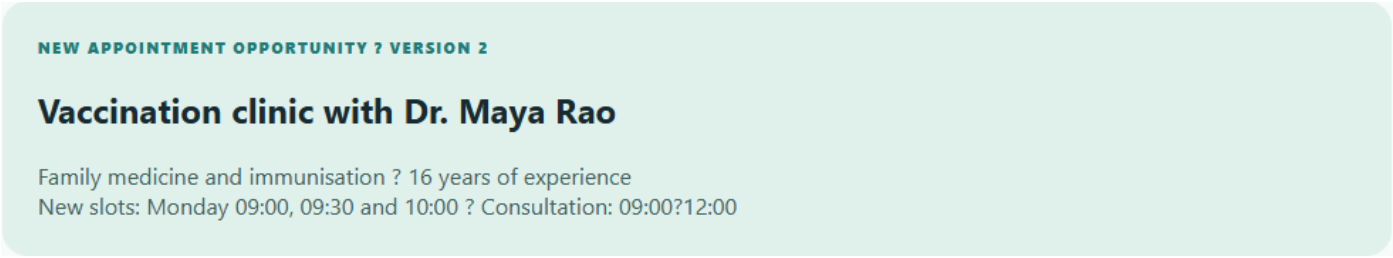


Step 7: Updating the hospital portal

The update adds a vaccination clinic with Dr. Maya Rao, consultation timings and three appointment slots. The GitHub commit was detected automatically by Jenkins SCM polling.

The prompt refers to a placement webpage in this step. The change was applied to the hospital portal, consistent with the Question 3 scenario.

Jenkins build #3 was started by an SCM change and deployed the updated source successfully.



Automatic SCM-triggered build:



Result: The updated application was deployed successfully and verified through its assigned NodePort.